

Drone Racing Algorithm and Strategy Summary

Zhuohao Ni, Bruce (Yuqian) Zhang

1 PPO Algorithm

The foundational aspect of our agent is the PPO algorithm implemented in `ppo.py`. We utilized the standard clipped-surrogate objective alongside an entropy maximization bonus ($c_{\text{ent}} = 0.005$) to sustain critical exploration during the highly volatile early training phases.

Adaptive Learning Rate: We observed late-stage policy collapses (around 2500 iterations) in early prototype models. To stabilize convergence, we used an adaptive learning rate schedule that scales based on the approximate KL divergence. If KL exceeds $2\times$ the target ($\bar{d}_{\text{KL}} = 0.01$), the learning rate halves; if it falls cleanly below half the target, it scales upward by $1.5\times$.

Vectorized Rollout Storage: To accelerate hyperparameter iteration speed and enable extremely large batch training (16384 parallel environments), we refactored Generalized Advantage Estimation (GAE) in `rollout_storage.py`. TD errors (δ_t) are entirely computed in a single GPU-parallelized pass. This limits the expensive, sequential Python operation solely to the recursive advantage derivation, lowering kernel deployment latency.

Configuration: We used an Actor network [256, 256] and a Critic network [512, 256, 128, 128] with ELU activations. Training used $\gamma = 0.99$ and $\lambda = 0.95$.

2 Observation Space (31 dims)

The policy receives a 31-dimensional ego-centric observation vector, with all spatial quantities expressed in the drone’s body frame:

Key design choice: replacing the 4D quaternion with a 3D gravity vector reduces dimensionality while preserving tilt information. The two-gate lookahead enables the policy to anticipate turns before arriving at the current gate.

3 Reward Shaping

Our reward configuration evolved significantly to overcome systemic collapse points. In developmental versions V3–V6, we attempted to enforce strict heading alignment penalties. Unintentionally, this overwhelmingly constrained the drone’s racing line, causing it to crash whenever it tried to physically decelerate for the tight turns between Gates 3 and 4. We reverted to a more balanced baseline (V2) that permitted aggressive,

Component	Dim	Description
Kinematics	6	Body-frame linear and angular velocities.
Orientation	3	Gravity vector projected into the body frame.
Lookahead Gates	9	Current, next, and next-next gate positions (body frame).
Gate Normals	6	Entry direction vectors for the current and next gates (body frame).
Gate Index	1	Normalized gate index (to contextualize powerloop vs. chicanes).
Alignment	2	sin and cos of the yaw error relative to the target gate.
Action History	4	Previous action vector used to penalize jitter.

drift-like cornering while incorporating the following primary elements:

Component	Scale	Strategic Intent
Gate Pass	200.0	Heavily prioritizes correct progression.
Progress	50.0	Δ distance to the target center.
Racing-Line Vel	8.0	Guides the primary momentum vector.
Orientation Pen	-1.0	Curbs flipping; abruptly activates if $ roll + pitch > 0.5 \text{ rad } (\sim 30^\circ)$.
Smoothness	-0.2	L_2 norm of consecutive actions.
Crash & Death	-11.0	Penalizes combined collisions (-1.0) and terminal resets (-10.0).

Corner Clipping: Instead of hard-coding complex geometric spline paths for the drone to follow, we modified the **Racing-Line Velocity** reward. For standard sequential gates, the target directional vector smoothly blends the relative angle of the current gate (5/8) with the next gate (3/8). This incentivizes the multi-layer perceptron policy to intuitively clip the inside of corners automatically, yielding highly efficient, curving racing lines. For the uniquely demanding Powerloop (Gate 3), it relies purely (1/1) on its immediate, current-gate velocity matrix.

4 Powerloop Bypass Strategy (Gates 2 & 3)

The central overlapping double-gate structure was originally meant to force a full vertical Powerloop. We initially built a 3-phase virtual target system to guide the drone up and over the top. However, during testing, we found that abandoning the vertical loop and taking a tighter path around the *side* of the structure was much faster.

We implemented this with a 2-phase virtual target bypass:

1. **Gate 2 Climb:** When targeting Gate 2, the target is overridden to an artificial *apex* $[0, -0.3, 1.6]$ m. The purpose of this apex is simply to pull the drone upwards early. It naturally clips through Gate 2 while it is already climbing, smoothly setting it up for the next maneuver without having to aim horizontally at the gate center first.
2. **Gate 3 Side Bypass:** Once the drone’s altitude passes $z = 1.3$ m or it gets within 0.8 m of the apex, the target shifts to an *offset target* $[0.425, 0, 0.75]$ m. This causes the drone to dive past the side of the Gate 3 structure and straight through the gate opening, skipping the slow "over-the-top" loop.

During earlier tests, some policies learned to take sub-optimal trajectories or pass backwards through Gate 2. While we experimented with strict termination rules for backward or wrong-side passes, we ultimately decided *against* using them. We found that allowing the drone to make these mistakes without instantly crashing resulted in much smoother and more stable training.

5 Reset Strategy

Episode initialization is designed to maximize training coverage across all starting conditions the policy might encounter during evaluation.

6 Sim-to-Real Domain Transfer

To successfully bridge the gap between simulation and the true evaluation dynamics, we heavily utilized Domain Randomization (DR). On every episode reset during training, all physical parameters (thrust-to-weight ratio, aerodynamic drag coefficients, PID controller gains) are re-sampled from uniform distributions.

Extreme Bounds: In our final fine-tuning run (V30), we randomized the physics parameters far beyond the official TA evaluation bounds. We scaled thrust-to-weight ratio by $\pm 8\%$, multiplied aerodynamic drag by $0.3 \times -2.5\times$, and widened PID variations to $\pm 40\%$. During evaluation, parameters are set to nominal defaults so the TA’s evaluation script can cleanly apply its own fixed perturbations.

Condition		Implementation Details
Starting Gate		Randomly selects one of the 7 gates as the target, ensuring all track segments are trained equally.
Spawn Position		Spawns behind the target gate at a distance that ramps from $[1.0, 2.0]$ m to $[1.5, 4.0]$ m (over 800 iters). Adds lateral (± 1.0 m), vertical (± 0.5 m), and yaw (± 0.3 rad) noise. Altitude clamped > 0.15 m.
Mid-track Spawns (40%)		Places the drone 20–80% of the way between consecutive gates. Training gate-to-gate transitions here was our biggest breakthrough, raising 3-lap success rates from 86.7% to 96.3%.
Ground Spawns ($\sim 20\%$)		Two mechanisms ($\sim 10\%$ each) trigger a ground spawn at $z = 0.05$ m (one with current velocity, one with zero velocity) to mirror the TA evaluation’s ground-start condition.
Velocity (50%)	Init	Half of all air spawns start with 0.5–3.0 m/s forward velocity to simulate mid-flight momentum. Ground spawns always start at zero velocity.

7 Fine-Tuning & Results

Our final model was trained in two stages. First, we trained a strong baseline (V26) for 5,000 iterations. Then, we fine-tuned it (creating V30) for an additional 2,000 iterations using the extreme Domain Randomization bounds described above and a significantly lowered entropy coefficient (0.001) to make the policy more deterministic.

We ran a paired evaluation testing 1,000 randomized environments using the official TA evaluation limits. The V30 fine-tuned model significantly outperformed the baseline, successfully repairing 78.5% of the random seeds where V26 previously failed or crashed.

Metric	V26 (Baseline)	V30 (Fine-Tuned)
Success Rate (SR)	94.9%	97.3%
Mean 3-Lap Time	15.78s	15.68s
Time Std Dev	0.54s	0.50s
Sub-16s Pass Rate	$\sim 82.0\%$	84.0%

By combining a body-frame observation space, blended velocity targets, a 2-phase side-bypass for the Powerloop, and aggressive domain randomization, our final policy is both very fast and highly resilient to dynamic disturbances.